

❑ Milestone 7 — Validation des saisies & Parsing des logs

Le support est une propriété de **Bechir Bejaoui**

Projet : Al Qalam Stock Manager —Papeterie tunisienne **Version** : 7.0.0 **Thème** : Expressions Régulières (re) **Dossier** : alqalamv7/

❑ Objectif du sprint

Introduire les **expressions régulières** (re) dans deux contextes complémentaires :

Contexte	Où	Ce qu'on fait
Validation formulaires	validators/regex_validators.py + dialogs.py	Patterns compilés, feedback ☐/☐ en temps réel
Parsing du journal	services/log_parser.py + AnalyseurFrame	Conversion en texte structuré, analyse par findall / finditer / MULTILINE

❑ Nouveaux fichiers V7

```
alqalamv7/
├── validators/
│   ├── __init__.py
│   └── regex_validators.py    ← NEW — patterns compilés + 5 fonctions de validation
├── services/
│   └── log_parser.py         ← NEW — formater journal → texte, analyser par regex
└── ui/frames/
    ├── analyseur_frame.py    ← NEW — 6ème onglet "🔍 Analyseur"
    └── dialogs.py            ← MAJ — validation regex en temps réel (<KeyRelease>)
```

Fichiers modifiés :

- config.py → APP_VERSION = "7.0.0"
- ui/app.py → 6ème onglet + arreter_polling() pour AnalyseurFrame
- main.py → commentaire V7

❑ Partie 1 —Le module validators/

regex_validators.py —patterns compilés

```
import re

REGEX = {
    # Référence produit : 2-6 MAJ + tiret + 1-5 alphanum MAJ
    # CRAY-001 PAP-A4 STYL-002 REG-001
    "ref": re.compile(r"^[A-Z]{2,6}-[A-Z0-9]{1,5}$"),

    # Nom : 2 à 60 caractères
    "nom": re.compile(r"^.{2,60}$"),

    # Prix TND : entier ou décimal (virgule ou point, 1-3 décimales)
    "prix": re.compile(r"^\d+([.,]\d{1,3})?$"),

    # Quantité / seuil : entier ≥ 0
    "qte": re.compile(r"^\d+$"),

    # Note : optionnelle, max 200 caractères
    "note": re.compile(r"^.{0,200}$", re.DOTALL),
}
```

Pourquoi re.compile() ?

```
# Sans compilation — Python repasse le pattern à CHAQUE appel
# Problème : <KeyRelease> est déclenché à chaque touche → appels très fréquents
```

```
for frappe in frappes_utilisateur:
    result = re.match(r"^[A-Z]{2,6}-[A-Z0-9]{1,5}$", frappe) # repasse à chaque fois
```

```
# Avec compilation — pattern analysé UNE SEULE FOIS
REF_RE = re.compile(r"^[A-Z]{2,6}-[A-Z0-9]{1,5}$")
```

```
for frappe in frappes_utilisateur:
    result = REF_RE.match(frappe) # réutilise la même regex compilée ☑
```

Fonctions de validation

Chaque fonction retourne un tuple (valide: bool, message: str):

```
def valider_ref(texte: str) -> tuple[bool, str]:
    t = texte.strip().upper()
    if not t:
        return False, "La référence est obligatoire."
    if REGEX["ref"].match(t):
        return True, "☑ Référence valide"
```

```

        return False, "Format : 2-6 lettres MAJ + tiret + 1-5 alphanum (ex: CRAY-001)"

def valider_prix(texte: str) -> tuple[bool, str]:
    t = texte.strip()
    if not t:
        return False, "Le prix est obligatoire."
    if REGEX["prix"].match(t):
        return True, "Prix valide"
    return False, "Format attendu : nombre entier ou décimal (ex: 1.500 ou 12,90)"

```

Tests de la regex référence :

```

from validators.regex_validators import valider_ref

print(valider_ref("CRAY-001")) # (True, "Référence valide")
print(valider_ref("PAP-A4")) # (True, "Référence valide")
print(valider_ref("cray-001")) # (False, "Format : ...") ← minuscules rejetées
print(valider_ref("REF-123456")) # (False, "Format : ...") ← 6 chiffres > max 5
print(valider_ref("")) # (False, "La référence est obligatoire.")

```

▣ Partie 2 —Validation en temps réel dans les dialogues

Le principe <KeyRelease> + regex

```

# Dans DialogueProduit._construire()
self.e_ref = ctk.CTkEntry(cadre, placeholder_text="ex: CRAY-001", height=35)
self.e_ref.pack(fill="x")

# Label d'indicateur sous le champ
self.lbl_ref = ctk.CTkLabel(cadre, text="", anchor="w",
                           font=ctk.CTkFont(size=10), text_color="#95A5A6")
self.lbl_ref.pack(fill="x")

# ← Liaison clé : chaque frappe appelle _valider_champs()
self.e_ref.bind("<KeyRelease>", self._valider_champs)

```

Callback de validation

```

def _valider_champs(self, _event=None):
    """Valide tous les champs par regex à chaque frappe."""
    ref_txt = self.e_ref.get().strip().upper()
    pa_txt = self.e_prix_achat.get().strip()
    # ... récupération de tous les champs

    resultats = [

```

```

        (valider_ref(ref_txt),    self.lbl_ref),
        (valider_prix(pa_txt),   self.lbl_prix_achat),
        # ...
    ]

    tous_valides = True
    for (valide, msg), lbl in resultats:
        lbl.configure(
            text=msg,
            text_color="#27AE60" if valide else "#E74C3C",    # vert / rouge
        )
        if not valide:
            tous_valides = False

    # Le bouton "Ajouter" s'active uniquement si TOUT est valide
    self.btnajouter.configure(
        state="normal" if tous_valides else "disabled",
        fg_color="#27AE60" if tous_valides else "#7F8C8D",
    )

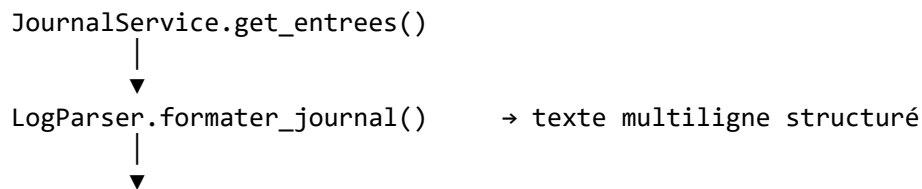
```

Comportement visuel

État du champ	Couleur indicateur	Texte indicateur
Vide (pas encore saisi)	Gris	(vide)
En cours de frappe — valide	Vert	☑ Référence valide
En cours de frappe — invalide	Rouge	Format : 2-6 lettres MAJ + tiret + ...
Tous valides	Bouton vert activé	☑ Ajouter
Un champ invalide	Bouton grisé désactivé	☑ Ajouter (grisé)

▣ Partie 3 —LogParser : analyse du journal par regex

Flux de données



```

LogParser.analyser()          → StatutLog (stats extraites par regex)
├── re.findall(r"␣", texte)    → nb_succes
├── re.findall(r"␣", texte)    → nb_erreurs
├── re.finditer(pattern_op, texte) → par_operation (Counter)
└── REGEX["ref_dans_texte"].findall → refs_actives (top 5)

```

Format du log généré

```

[2025-03-09 10:42:01] [entree  ] ␣ CRAY-001 | Entrée stock | +50 unités
[2025-03-09 10:43:15] [sortie  ] ␣ STYL-001 | Sortie stock | -
5 unités
[2025-03-09 10:44:00] [ajout   ] ␣ NEW-001   | Nouveau produit | ref=NEW-
001
[2025-03-09 10:45:00] [sortie  ] ␣ REG-001   | Sortie stock   | stock insuffisant

```

Méthode analyser() —regex en action

```

def analyser(self) -> StatutLog:
    texte = self.formater_journal()

    # re.findall() — trouve TOUTES les occurrences d'un pattern
    nb_succes = len(re.findall(r"␣", texte))
    nb_erreurs = len(re.findall(r"␣", texte))

    # re.findall() + re.MULTILINE — lignes contenant ␣
    # ^ et $ ancrent sur chaque LIGNE grâce à re.MULTILINE
    lignes_erreur = re.findall(r"^\.*␣.*$", texte, re.MULTILINE | re.UNICODE)

    # re.finditer() avec groupe nommé (?P<op>...) — extraire les types d'opération
    pattern_op = re.compile(r"\[(?P<op>[a-z]+)\s*\]", re.IGNORECASE)
    operations = [m.group("op") for m in pattern_op.finditer(texte)]
    par_operation = Counter(operations) # {"entree": 5, "sortie": 3, ...}

    # Pattern pré-compilé — REGEX["ref_dans_texte"] = r"\b[A-Z]{2,6}-[A-Z0-9]{1,5}\b"
    toutes_refs = REGEX["ref_dans_texte"].findall(texte)
    refs_actives = [ref for ref, _ in Counter(toutes_refs).most_common(5)]

    return StatutLog(
        nb_total=nb_succes + nb_erreurs,
        nb_erreurs=nb_erreurs,
        nb_succes=nb_succes,
        par_operation=par_operation,
        refs_actives=refs_actives,
        lignes_erreur=lignes_erreur,
    )

```

```

        texte_brut=texte,
    )

```

Recherche interactive par regex

```

def rechercher(self, motif: str, ignorer_casse: bool = True) -> list[str]:
    """
    Filtre les lignes du journal par regex.
    re.MULTILINE : ^ et $ ancrent sur chaque ligne du texte.
    """
    texte = self.formater_journal()
    flags = re.MULTILINE | re.UNICODE
    if ignorer_casse:
        flags |= re.IGNORECASE

    try:
        # Construit un pattern qui sélectionne les lignes contenant le motif
        pattern = re.compile(rf"^{motif}.*$", flags)
        return pattern.findall(texte)
    except re.error:
        # Motif invalide (ex: "abc") → affiche tout sans planter
        return texte.splitlines()

```

Exemples de recherches :

```

parser.rechercher("CRAY")           # Lignes contenant "CRAY" (insensible casse)
parser.rechercher("CRAY|STYL")       # Lignes contenant "CRAY" OU "STYL" — alternance
parser.rechercher("entree")          # Lignes de type entrée
parser.rechercher(r"[-]\d+")         # Lignes avec une quantité négative (sorties)
parser.rechercher("[\u0019]", ignorer_casse=False) # uniquement les lignes en erreur

```

▣ Partie 4 —AnalyseurFrame —le 6ème onglet

Architecture de l'onglet

🔍 Analyseur de journal — Expressions régulières [V7]		
🔍 Statistiques Total : 12 Succès : 11 Erreurs : 1	🔍 Recherche regex : [__CRAY STYL__] 🔍	
	🔍 Insensible casse	
	🔍 Journal formaté 8 ligne(s)	
	[2025-03-09 10:42] [entree] 🔍 CRAY-	
001 ...		

	Répartition :	[2025-03-09 10:43] [sortie] ☐ CRAY-
001	...	
	entree : 5	[2025-03-09 10:44] [entree] ☐ STYL-
001	...	
	sortie : 4	...
	ajout : 2	
	ajustement: 1	
	Refs actives :	
	CRAY-001	
	STYL-001	
	☐ Filtres rapides	
	[Tout afficher]	
	[☐ Entrées]	
	[☐ Sorties]	
	[☐ Ajustements]	
	[☐ Erreurs seul.]	
	[☐ Rafraîchir]	

Validation de la regex saisie

```
def _recherche_live(self, _event=None):
    motif = self.e_recherche.get().strip()
    ignorer_casse = bool(self.chk_casse.get())

    # Vérifier la validité syntaxique de la regex AVANT de l'utiliser
    try:
        re.compile(motif)
        self.lbl_regex_statut.configure(text="☐ regex valide", text_color="#27AE60")
    except re.error as exc:
        self.lbl_regex_statut.configure(
            text=f"☐ regex invalide : {exc}", text_color="#E74C3C",
        )
        return # ← ne pas appliquer une regex cassée

    lignes = self._parser.rechercher(motif, ignorer_casse=ignorer_casse)
    self._afficher_lignes(lignes)
```

☐ Tableau de synthèse —fonctions re utilisées en V7

Fonction	Où	Utilisation
<code>re.compile(pattern)</code>	<code>regex_validators.py</code>	Pré-compile les 5 patterns au démarrage
<code>REGEX["ref"].match(t)</code>	<code>valider_ref()</code>	Valide la référence en temps réel
<code>re.findall(r"█", texte)</code>	<code>log_parser.analyse()</code>	Compte les succès dans le log
<code>re.findall(r"^\.*█.*\$", texte, re.MULTILINE)</code>	<code>log_parser.analyse()</code>	Extrait les lignes d'erreur
<code>re.finditer(pattern_op, texte)</code>	<code>log_parser.analyse()</code>	Extrait les types d'opération avec groupe nommé
<code>REGEX["ref_dans_texte"]</code>	<code>log_parser.analyse()</code>	Extrait toutes les références
<code>re.compile(rf"^\.*{motif}.*\$", flags)</code>	<code>log_parser.recherche()</code>	Filtre les lignes par motif utilisateur
<code>re.error</code>	<code>analyseur_frame._recherche()</code>	Gère les regex invalides

☐ Critères de validation du milestone

Validation formulaires

- ☐ Taper CRAY-001 → indicateur **vert** ☐ Référence valide
- ☐ Taper cray-001 → indicateur **rouge** Format : 2-6 lettres MAJ...
- ☐ Taper 1.500 dans prix → indicateur **vert** ☐ Prix valide
- ☐ Taper -5 dans prix → indicateur **rouge** Format attendu...
- ☐ Bouton "Ajouter" **grisé** tant qu'un champ est invalide
- ☐ Bouton "Ajouter" **vert et actif** quand tout est valide

Analysateur de journal (après quelques opérations)

- ☐ L'onglet "☐ Analysateur" affiche les statistiques (total, succès, erreurs)
- ☐ La répartition par type est correcte
- ☐ La recherche CRAY filtre les lignes en temps réel
- ☐ La recherche CRAY|STYL (alternance regex) fonctionne
- ☐ Une regex invalide *abc affiche ☐ regex invalide sans planter
- ☐ Les boutons de filtre (☐ Entrées, ☐ Erreurs seul.) fonctionnent
- ☐ Les lignes ☐ sont en vert, les lignes ☐ sont en rouge

□ Lancement

```
cd alqalamv7  
py -3 main.py
```

Démonstration recommandée :

1. Ouvrir "Nouveau produit" → taper `cray-001` en minuscules → observer le rouge
 2. Corriger en `CRAY-002` → observer le vert et le bouton qui s'active
 3. Valider → faire quelques entrées/sorties
 4. Aller dans l'onglet "□ Analyseur"
 5. Rechercher `CRAY` → les lignes `CRAY` apparaissent
 6. Rechercher `□` → seules les erreurs apparaissent
 7. Taper `[*invalid` → message `□ regex invalide`
-

□ Points clés à retenir

re.compile() : toujours compiler les patterns réutilisés —essentiel pour les callbacks `<KeyRelease>` qui s'exécutent à chaque frappe.

re.MULTILINE : sans ce flag, `^` et `$` ancrent sur toute la chaîne — avec ce flag, ils ancrent sur chaque ligne. Indispensable pour filtrer des logs multiligne.

re.findall() vs re.finditer() : `findall` retourne une liste de chaînes (pratique pour compter), `finditer` retourne des Match objects (indispensable pour accéder aux groupes nommés (`?P<nom>...`)).

re.error : toujours capturer l'exception quand la regex vient de l'utilisateur —une regex malformée planterait sinon l'application.

Pattern rf"^.*{motif}.*\$" : f-string + raw string combinés — insère le motif utilisateur dans la regex dynamiquement.